Université de Jendouba
Institut Supérieur des Langues Appliquées et Informatique de Béja

Module:

Framework web Python



2MDS/ A.U. 2026-2027

Enseignant responsable: Nabil KHEMIRI, E-mail: nabil.khemiri@gmail.com

Chapitre 3 : Les sessions avec Flask

Définition d’une session

- **Une session** désigne **une période d'interaction active et continue** entre **un utilisateur** (ou un programme) et une application, **un serveur** ou un environnement d'exécution.

- **Le protocole HTTP est par nature « sans état »,** ce qui signifie que **le serveur oublie qui vous êtes d'une page à l'autre.**

- **Une session web** permet de **conserver des données spécifiques à un utilisateur** (comme un statut de connexion, un panier d'achat, ID, nom, prénom, email) **à travers plusieurs requêtes HTTP.**

Les sessions dans flask

- Les sessions dans Flask **stockent les données spécifiques à l'utilisateur à travers les requêtes**, **en utilisant des cookies.**

- **Les données sont stockées du côté du client**, mais **signées avec une clé secrète pour assurer la sécurité**.

- **Installation de flask-session :**

```
py -m pip install flask flask-session
```

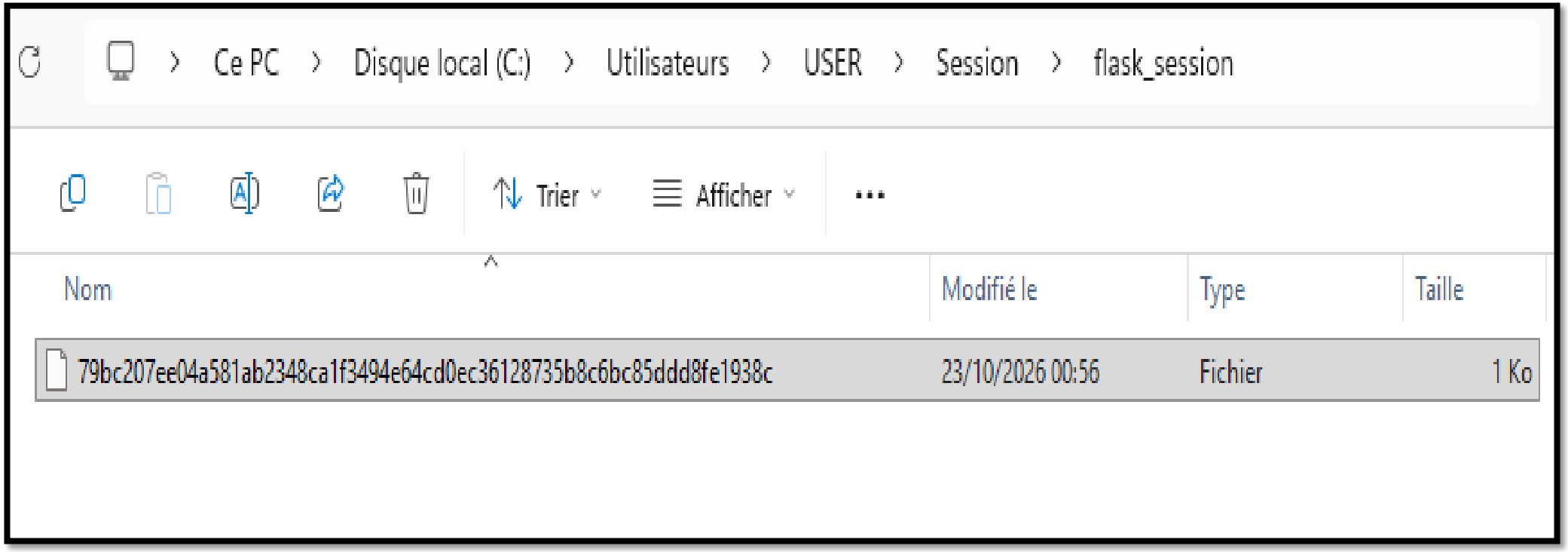
Les cookies

- **Un cookie** est un **fichier texte qu'un site web envoie et stocke sur votre appareil (ordinateur, tablette ou smartphone) via votre navigateur.**

- Un cookie **stocke les données des utilisateurs sous forme de paires de valeurs clés**.

- Permettant aux sites Web de **mémoriser les connexions, les préférences et d'autres détails.** Cela permet **d'améliorer l'expérience utilisateur** en rendant le site **plus pratique** et p**ersonnalisé.**

Fichier de session



`SESSION_TYPE = "filesystem"` : les données de session sont enregistrées dans un répertoire côté serveur.

`79bc207ee04a581ab2348ca1f3494e64cd0ec36128735b8c6bc85ddd8fe1938c` : Nom du fichier de session

Stockage des fichiers de session

```
# Stockage dans des fichiers
app.config["SESSION_TYPE"] = "filesystem"

# Stockage avec Redis
app.config["SESSION_TYPE"] = "redis"

# Stockage avec Memcached
app.config["SESSION_TYPE"] = "memcached"

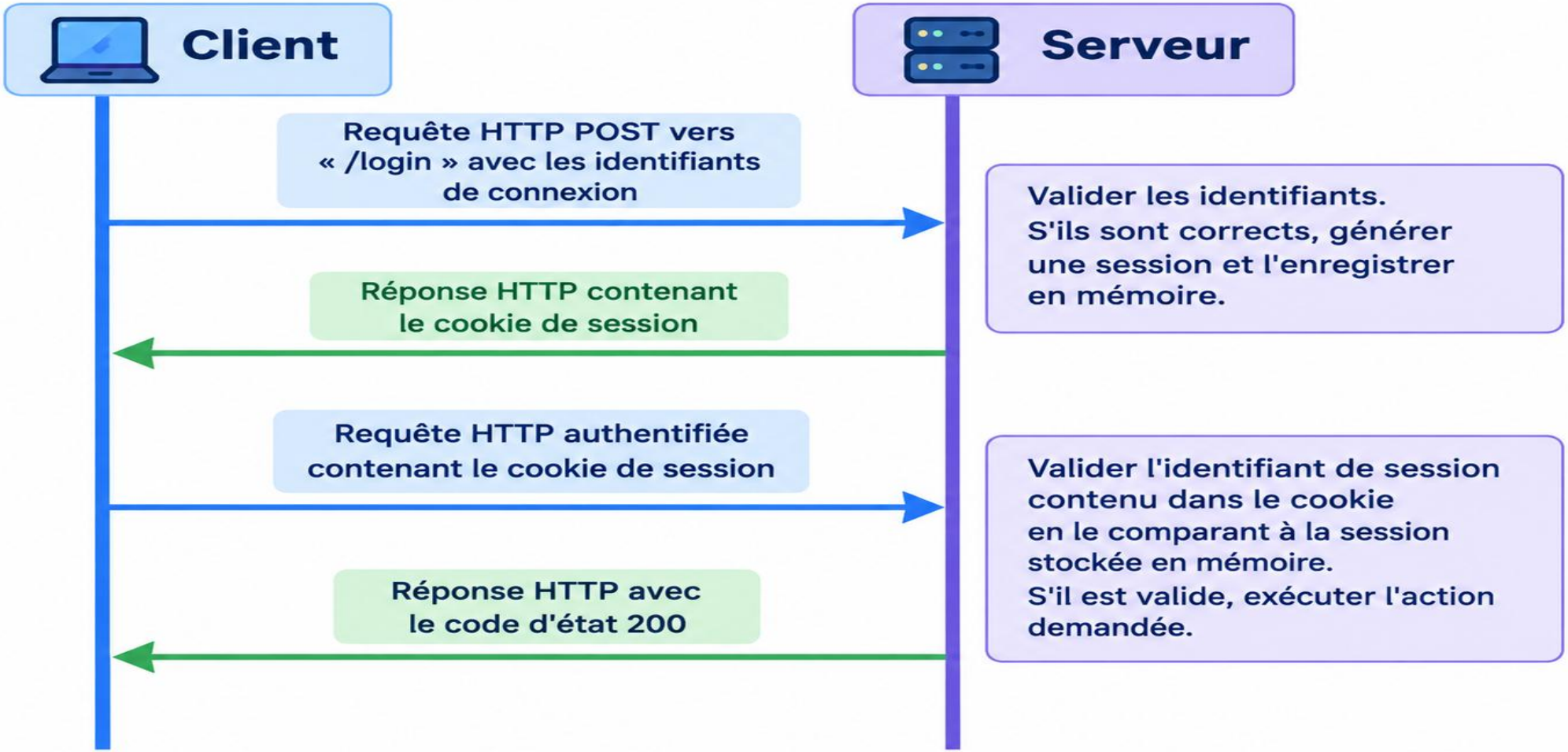
# Stockage dans une base SQL via SQLAlchemy
app.config["SESSION_TYPE"] = "sqlalchemy"

# Stockage dans MongoDB
app.config["SESSION_TYPE"] = "mongodb"

# Stockage avec CacheLib
app.config["SESSION_TYPE"] = "cachelib"

# Stockage dans Amazon DynamoDB
app.config["SESSION_TYPE"] = "dynamodb"
```

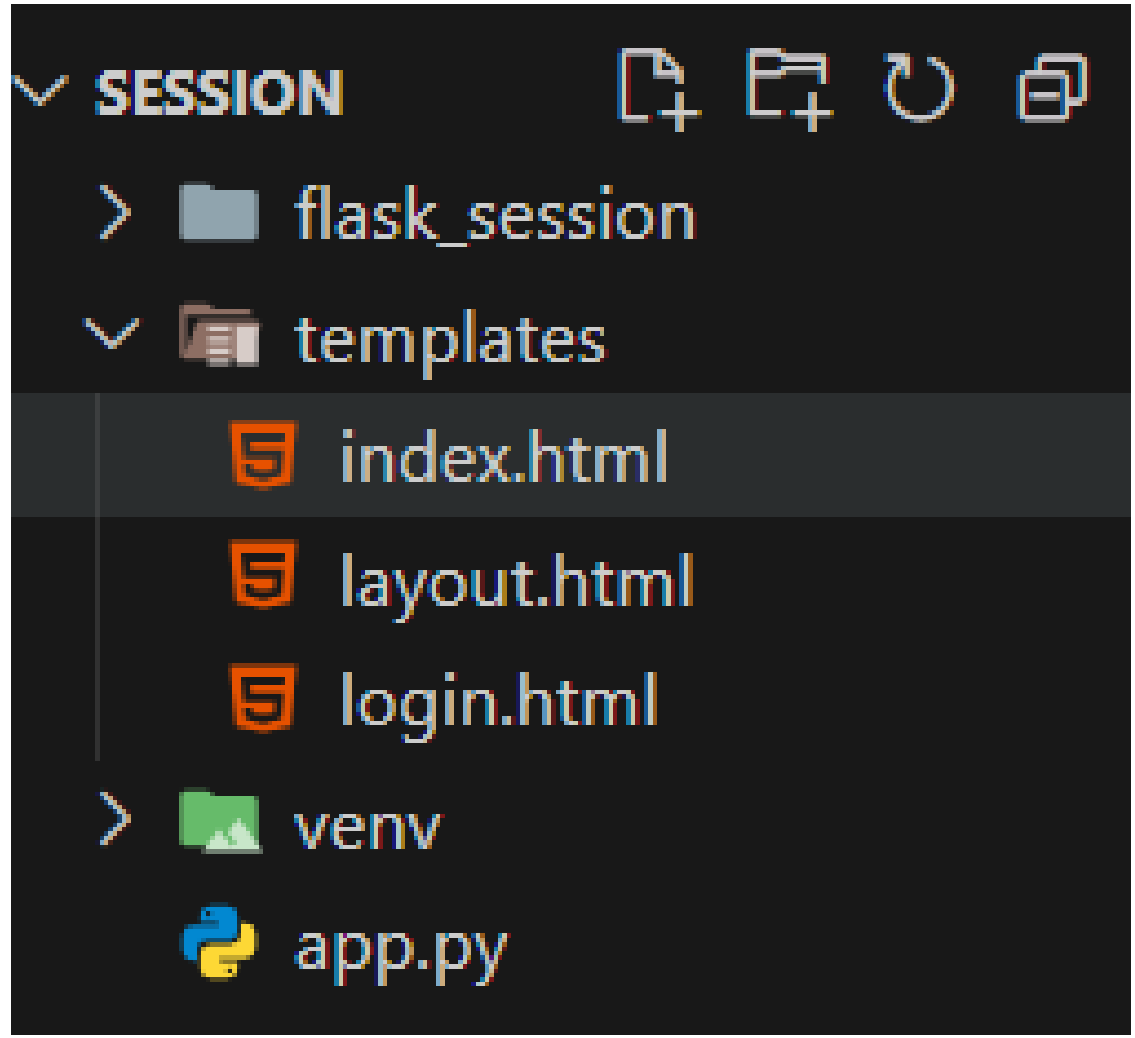
Communication client/serveur en utilisant une session



À retenir : le cookie de session permet au serveur de reconnaître le client lors des requêtes suivantes, sans lui demander de saisir à nouveau ses identifiants. Le code HTTP **200** indique que la requête a été traitée avec succès.

Exemple

Structure du projet



Layout.html

```
<!DOCTYPE html>
<html lang="fr">
    <head>
        <meta name="viewport" content="initial-scale=1, width=device-width">
        <title>Les sessions avec Flask</title>
    </head>
    <body>
        <!-- Déclare un bloc nommé content qui pourra être rempli
            par les pages qui héritent de ce template -->
            {% block content %}
        <!-- Fin du bloc content -->
            {% endblock %}
    </body>
</html>
```

login.html

```
1
2  <!-- Permet de réutiliser la structure HTML définie dans layout.html. -->
3  {% extends "layout.html" %}
4
5      <!-- Délimite le contenu spécifique de cette page. -->
6      {% block content %}
7          <h1>Inscription </h1>
8          <form action="/login" method="POST">
9              <label><b> Nom : </b></label><input placeholder="Votre nom" type="text" name="name" autofocus required>
10             <input type="submit" value="Envoyer">
11         </form>
12     {% endblock %}
13
14
```

index.html

```
{% extends "layout.html" %}

{% block content %}
    <!-- Vérifie si la variable "name" existe dans la session
      et contient une valeur -->
  {% if session.name %}
     <!-- Affiche la valeur de l'attribut "name" stocké dans la session -->
    <strong> Vous êtes enregistré(e) en tant que :  </strong> {{ session.name }}. <br>
     <!-- Affiche l'identifiant de session transmis par Flask
        au template sous le nom "session_id" -->
    <strong> Votre ID de session est  :  </strong> {{ session_id }}. <br><br>
    <form action="/logout" method="GET">
        <button type="submit">Se déconnecter</button>
    </form>

  {% else %}
    <strong>Vous n'êtes pas enregistré(e). </strong><a href="/login">Login</a>.
  {% endif %}
{% endblock %}

```

app.py

```python
# Importation de la classe Flask pour créer l'application web
# render_template permet d'afficher des pages HTML
# redirect permet de rediriger l'utilisateur vers une autre route
# request permet de récupérer les données envoyées par le navigateur
# session permet de stocker et de récupérer des données de session

from flask import Flask, render_template, redirect, request, session
from flask_session import Session
import uuid
import secrets

app = Flask(__name__)

# Configuration de Flask-Session
app.config["SECRET_KEY"] = secrets.token_hex() #Génère une clé aléatoire sécurisée.
app.config["SESSION_PERMANENT"] = False   #Désactive la durée de vie permanente de la session.
app.config["SESSION_TYPE"] = "filesystem" #Stocke les données de session dans des fichiers sur le serveur.

Session(app) #Active l'extension Flask-Session.
```

app.py (suite)

```
20
21  # Associe la fonction index() à l'URL racine "/"
22  @app.route("/")
23  def index():
24
25      # Vérifier si l'utilisateur est connecté
26      if not session.get("name"):
27          return redirect("/login")
28
29       # Si l'utilisateur est connecté, afficher la page index.html
30      # en transmettant les informations nécessaires au template
31      return render_template(
32          "index.html",
33          name=session["name"],
34          session_id=session["session_id"]
35      )
36
```

app.py (suite)

```python
# Autoriser les requêtes GET (affichage du formulaire)
# et POST (envoi des données du formulaire)

@app.route("/login", methods=["GET", "POST"])
def login():

    if request.method == "POST":

        # Récupérer le nom depuis le formulaire
        session["name"] = request.form.get("name")

        # Créer un identifiant unique une seule fois
        if "session_id" not in session:
            session["session_id"] = str(uuid.uuid4())

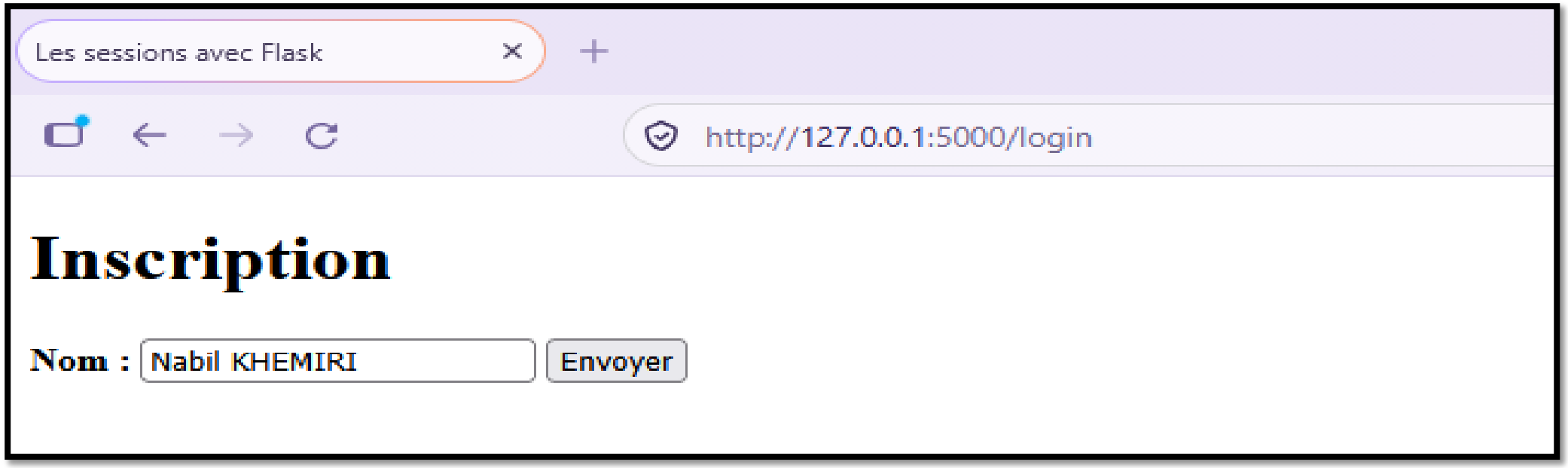
        # Rediriger l'utilisateur vers la page d'accueil "/"
        return redirect("/")
    # Si la requête est GET, afficher le formulaire de connexion
    return render_template("login.html")

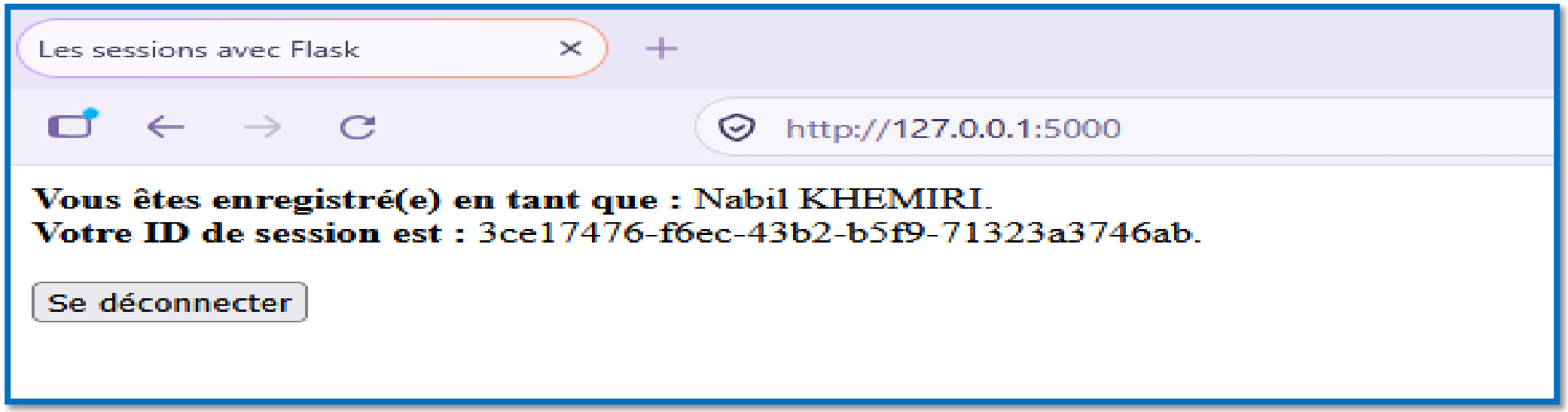
```

app.py (suite)

```
57  # Associer la fonction logout() à l'URL "/logout"
58  @app.route("/logout")
59  def logout():
60
61      # Supprimer toutes les données enregistrées dans la session
62      # Cela supprime notamment le nom de l'utilisateur et son identifiant de session
63      session.clear()
64      # Rediriger l'utilisateur vers la page de connexion "/login"
65      return redirect("/login")
66
67
68  if __name__ == "__main__":
69      app.run(debug=True)
70
```

Exécution


Les sessions avec Flask
http://127.0.0.1:5000/login
Inscription
Nom : Nabil KHEMIRI
Envoyer


Les sessions avec Flask
http://127.0.0.1:5000
Vous êtes enregistré(e) en tant que : Nabil KHEMIRI.
Votre ID de session est : 3ce17476-f6ec-43b2-b5f9-71323a3746ab.
Se déconnecter